

```
In [ ]: !jupyter nbconvert --to html d4_Regularization_w_L2_Dropout.ipynb
```

Continuous of CNN

```
In [9]: # Needed Libraries
import os
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
from tensorflow.keras.datasets import cifar10
from datetime import datetime
import time
# physical_devices = tf.config.list_physical_devices('GPU')
# if physical_devices:
#     tf.config.experimental.set_memory_growth(physical_devices[0], True)
#     print("Memory growth set for GPU.")
# else:
#     print("No GPU found. Running on CPU.")
```

```
In [10]: (x_train, y_train), (x_test, y_test) = cifar10.load_data()
```

```
In [11]: x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

```
In [12]: #same model according to Function base
print("Starting of the function Datetime: ", datetime.now())

def my_model():
    inputs = keras.Input(shape=(32,32,3))
    x = layers.Conv2D(
        32,3, padding='same', kernel_regularizer=regularizers.l2(0.01) #adding regul
    )(inputs)
    x = layers.BatchNormalization()(x)
    x = keras.activations.relu(x)
    x = layers.MaxPooling2D()(x)
    x = layers.Conv2D(
        64, 5, padding='same', kernel_regularizer=regularizers.l2(0.01) #adding reg
    )(x)
    x = layers.BatchNormalization()(x)
    x = keras.activations.relu(x)
    x = layers.Conv2D(
        128,3, padding='same', kernel_regularizer=regularizers.l2(0.01) #adding regu
    )(x)
    x = layers.BatchNormalization()(x)
    x = keras.activations.relu(x)
    x = layers.Flatten()(x)
    x = layers.Dense(
        64, activation='relu', kernel_regularizer=regularizers.l2(0.01))(x)
    x = layers.Dropout(0.5)(x)
    outputs = layers.Dense(10)(x)
    model1 = keras.Model(inputs=inputs, outputs=outputs)
```

```
    return model1

model1 = my_model()
# print(my_model())
# print(model())
print(model1.summary())
print("In between of the function Datetime: ",datetime.now())

model1.compile(
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=keras.optimizers.Adam(learning_rate=3e-4),
    metrics=["accuracy"],
)

model1.fit(x_train, y_train, batch_size=64, epochs=150, verbose=2)
model1.evaluate(x_test, y_test, batch_size=64, verbose=2)

print("Ending of the function Datetime: ",datetime.now())
```

Starting of the function Datetime: 2025-07-30 13:10:08.502162

Model: "model_1"

Layer (type)	Output Shape	Param #
=====		
input_2 (InputLayer)	[(None, 32, 32, 3)]	0
conv2d_3 (Conv2D)	(None, 32, 32, 32)	896
batch_normalization_3 (Batch Normalization)	(None, 32, 32, 32)	128
tf.nn.relu_3 (TFOpLambda)	(None, 32, 32, 32)	0
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_4 (Conv2D)	(None, 16, 16, 64)	51264
batch_normalization_4 (Batch Normalization)	(None, 16, 16, 64)	256
tf.nn.relu_4 (TFOpLambda)	(None, 16, 16, 64)	0
conv2d_5 (Conv2D)	(None, 16, 16, 128)	73856
batch_normalization_5 (Batch Normalization)	(None, 16, 16, 128)	512
tf.nn.relu_5 (TFOpLambda)	(None, 16, 16, 128)	0
flatten_1 (Flatten)	(None, 32768)	0
dense_2 (Dense)	(None, 64)	2097216
dropout (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 10)	650

=====

Total params: 2224778 (8.49 MB)

Trainable params: 2224330 (8.49 MB)

Non-trainable params: 448 (1.75 KB)

None

In between of the function Datetime: 2025-07-30 13:10:08.697457

Epoch 1/150

782/782 - 157s - loss: 3.0994 - accuracy: 0.1753 - 157s/epoch - 201ms/step

Epoch 2/150

782/782 - 139s - loss: 2.1791 - accuracy: 0.2314 - 139s/epoch - 177ms/step

Epoch 3/150

782/782 - 135s - loss: 2.0324 - accuracy: 0.2437 - 135s/epoch - 173ms/step

Epoch 4/150

782/782 - 153s - loss: 1.9897 - accuracy: 0.2525 - 153s/epoch - 195ms/step

Epoch 5/150

782/782 - 133s - loss: 1.9675 - accuracy: 0.2601 - 133s/epoch - 170ms/step

Epoch 6/150
782/782 - 134s - loss: 1.9485 - accuracy: 0.2654 - 134s/epoch - 171ms/step
Epoch 7/150
782/782 - 139s - loss: 1.9247 - accuracy: 0.2770 - 139s/epoch - 178ms/step
Epoch 8/150
782/782 - 138s - loss: 1.9196 - accuracy: 0.2858 - 138s/epoch - 177ms/step
Epoch 9/150
782/782 - 139s - loss: 1.9003 - accuracy: 0.2941 - 139s/epoch - 178ms/step
Epoch 10/150
782/782 - 138s - loss: 1.8876 - accuracy: 0.3040 - 138s/epoch - 177ms/step
Epoch 11/150
782/782 - 147s - loss: 1.8786 - accuracy: 0.3103 - 147s/epoch - 189ms/step
Epoch 12/150
782/782 - 145s - loss: 1.8633 - accuracy: 0.3197 - 145s/epoch - 185ms/step
Epoch 13/150
782/782 - 147s - loss: 1.8603 - accuracy: 0.3297 - 147s/epoch - 188ms/step
Epoch 14/150
782/782 - 144s - loss: 1.8462 - accuracy: 0.3372 - 144s/epoch - 184ms/step
Epoch 15/150
782/782 - 132s - loss: 1.8362 - accuracy: 0.3374 - 132s/epoch - 169ms/step
Epoch 16/150
782/782 - 129s - loss: 1.8236 - accuracy: 0.3429 - 129s/epoch - 164ms/step
Epoch 17/150
782/782 - 135s - loss: 1.8173 - accuracy: 0.3435 - 135s/epoch - 172ms/step
Epoch 18/150
782/782 - 128s - loss: 1.8112 - accuracy: 0.3438 - 128s/epoch - 163ms/step
Epoch 19/150
782/782 - 130s - loss: 1.8074 - accuracy: 0.3457 - 130s/epoch - 167ms/step
Epoch 20/150
782/782 - 137s - loss: 1.7985 - accuracy: 0.3495 - 137s/epoch - 175ms/step
Epoch 21/150
782/782 - 130s - loss: 1.7901 - accuracy: 0.3539 - 130s/epoch - 167ms/step
Epoch 22/150
782/782 - 130s - loss: 1.7903 - accuracy: 0.3518 - 130s/epoch - 166ms/step
Epoch 23/150
782/782 - 129s - loss: 1.7898 - accuracy: 0.3490 - 129s/epoch - 164ms/step
Epoch 24/150
782/782 - 131s - loss: 1.7791 - accuracy: 0.3555 - 131s/epoch - 167ms/step
Epoch 25/150
782/782 - 130s - loss: 1.7834 - accuracy: 0.3536 - 130s/epoch - 166ms/step
Epoch 26/150
782/782 - 128s - loss: 1.7800 - accuracy: 0.3563 - 128s/epoch - 163ms/step
Epoch 27/150
782/782 - 144s - loss: 1.7733 - accuracy: 0.3562 - 144s/epoch - 184ms/step
Epoch 28/150
782/782 - 130s - loss: 1.7715 - accuracy: 0.3576 - 130s/epoch - 166ms/step
Epoch 29/150
782/782 - 131s - loss: 1.7695 - accuracy: 0.3614 - 131s/epoch - 167ms/step
Epoch 30/150
782/782 - 136s - loss: 1.7687 - accuracy: 0.3613 - 136s/epoch - 173ms/step
Epoch 31/150
782/782 - 134s - loss: 1.7665 - accuracy: 0.3617 - 134s/epoch - 171ms/step
Epoch 32/150
782/782 - 134s - loss: 1.7625 - accuracy: 0.3622 - 134s/epoch - 171ms/step
Epoch 33/150
782/782 - 136s - loss: 1.7596 - accuracy: 0.3616 - 136s/epoch - 174ms/step

Epoch 34/150
782/782 - 141s - loss: 1.7551 - accuracy: 0.3669 - 141s/epoch - 181ms/step
Epoch 35/150
782/782 - 133s - loss: 1.7569 - accuracy: 0.3650 - 133s/epoch - 171ms/step
Epoch 36/150
782/782 - 145s - loss: 1.7602 - accuracy: 0.3643 - 145s/epoch - 186ms/step
Epoch 37/150
782/782 - 136s - loss: 1.7514 - accuracy: 0.3684 - 136s/epoch - 174ms/step
Epoch 38/150
782/782 - 131s - loss: 1.7470 - accuracy: 0.3692 - 131s/epoch - 168ms/step
Epoch 39/150
782/782 - 143s - loss: 1.7463 - accuracy: 0.3698 - 143s/epoch - 183ms/step
Epoch 40/150
782/782 - 142s - loss: 1.7454 - accuracy: 0.3687 - 142s/epoch - 181ms/step
Epoch 41/150
782/782 - 141s - loss: 1.7379 - accuracy: 0.3738 - 141s/epoch - 180ms/step
Epoch 42/150
782/782 - 133s - loss: 1.7387 - accuracy: 0.3733 - 133s/epoch - 170ms/step
Epoch 43/150
782/782 - 137s - loss: 1.7424 - accuracy: 0.3713 - 137s/epoch - 175ms/step
Epoch 44/150
782/782 - 140s - loss: 1.7386 - accuracy: 0.3719 - 140s/epoch - 179ms/step
Epoch 45/150
782/782 - 134s - loss: 1.7384 - accuracy: 0.3706 - 134s/epoch - 172ms/step
Epoch 46/150
782/782 - 138s - loss: 1.7423 - accuracy: 0.3704 - 138s/epoch - 176ms/step
Epoch 47/150
782/782 - 129s - loss: 1.7371 - accuracy: 0.3727 - 129s/epoch - 166ms/step
Epoch 48/150
782/782 - 130s - loss: 1.7279 - accuracy: 0.3734 - 130s/epoch - 167ms/step
Epoch 49/150
782/782 - 132s - loss: 1.7421 - accuracy: 0.3724 - 132s/epoch - 169ms/step
Epoch 50/150
782/782 - 131s - loss: 1.7311 - accuracy: 0.3764 - 131s/epoch - 167ms/step
Epoch 51/150
782/782 - 126s - loss: 1.7258 - accuracy: 0.3767 - 126s/epoch - 162ms/step
Epoch 52/150
782/782 - 135s - loss: 1.7315 - accuracy: 0.3782 - 135s/epoch - 172ms/step
Epoch 53/150
782/782 - 137s - loss: 1.7286 - accuracy: 0.3766 - 137s/epoch - 175ms/step
Epoch 54/150
782/782 - 142s - loss: 1.7328 - accuracy: 0.3738 - 142s/epoch - 182ms/step
Epoch 55/150
782/782 - 137s - loss: 1.7273 - accuracy: 0.3773 - 137s/epoch - 175ms/step
Epoch 56/150
782/782 - 150s - loss: 1.7316 - accuracy: 0.3766 - 150s/epoch - 192ms/step
Epoch 57/150
782/782 - 154s - loss: 1.7270 - accuracy: 0.3808 - 154s/epoch - 197ms/step
Epoch 58/150

KeyboardInterrupt

Traceback (most recent call last)

Cell In[12], line 42

```
34 print("In between of the function Datetime: ",datetime.now())
36 model1.compile(
37     loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
38     optimizer=keras.optimizers.Adam(learning_rate=3e-4),
39     metrics=["accuracy"],
40 )
---> 42 model1.fit(x_train, y_train, batch_size=64, epochs=150, verbose=2)
43 model1.evaluate(x_test, y_test, batch_size=64, verbose=2)
45 print("Ending of the function Datetime: ",datetime.now())
```

File ~\miniconda3\envs\myenv\lib\site-packages\keras\src\utils\traceback_utils.py:65, in filter_traceback.<locals>.error_handler(*args, **kwargs)

```
63 filtered_tb = None
64 try:
---> 65     return fn(*args, **kwargs)
66 except Exception as e:
67     filtered_tb = _process_traceback_frames(e.__traceback__)
```

File ~\miniconda3\envs\myenv\lib\site-packages\keras\src\engine\training.py:1742, in Model.fit(self, x, y, batch_size, epochs, verbose, callbacks, validation_split, validation_data, shuffle, class_weight, sample_weight, initial_epoch, steps_per_epoch, validation_steps, validation_batch_size, validation_freq, max_queue_size, workers, use_multiprocessing)

```
1734 with tf.profiler.experimental.Trace(
1735     "train",
1736     epoch_num=epoch,
1737     (...)
1739     _r=1,
1740 ):
1741     callbacks.on_train_batch_begin(step)
-> 1742     tmp_logs = self.train_function(iterator)
1743     if data_handler.should_sync:
1744         context.async_wait()
```

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\util\traceback_utils.py:150, in filter_traceback.<locals>.error_handler(*args, **kwargs)

```
148 filtered_tb = None
149 try:
--> 150     return fn(*args, **kwargs)
151 except Exception as e:
152     filtered_tb = _process_traceback_frames(e.__traceback__)
```

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\polymorphic_function\polymorphic_function.py:825, in Function.__call__(self, *args, **kws)

```
822 compiler = "xla" if self._jit_compile else "nonXla"
824 with OptionalXlaContext(self._jit_compile):
--> 825     result = self._call(*args, **kws)
827 new_tracing_count = self.experimental_get_tracing_count()
828 without_tracing = (tracing_count == new_tracing_count)
```

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\polymorphic_function\polymorphic_function.py:857, in Function._call(self, *args, **kws)

```
854     self._lock.release()
```

```

855 # In this case we have created variables on the first call, so we run the
856 # defunned version which is guaranteed to never create variables.
--> 857 return self._no_variable_creation_fn(*args, **kws) # pylint: disable=not
-callable
858 elif self._variable_creation_fn is not None:
859 # Release the lock early so that multiple threads can perform the call
860 # in parallel.
861 self._lock.release()

```

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\polymorphic_function\tracing_compiler.py:148, in TracingCompiler.__call__(self, *args, **kwargs)

```

145 with self._lock:
146 (concrete_function,
147 filtered_flat_args) = self._maybe_define_function(args, kwargs)
--> 148 return concrete_function._call_flat(
149 filtered_flat_args, captured_inputs=concrete_function.captured_inputs)

```

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\polymorphic_function\monomorphic_function.py:1349, in ConcreteFunction._call_flat(self, args, captured_inputs)

```

1345 possible_gradient_type = gradients_util.PossibleTapeGradientTypes(args)
1346 if (possible_gradient_type == gradients_util.POSSIBLE_GRADIENT_TYPES_NONE
1347 and executing_eagerly):
1348 # No tape is watching; skip to running the function.
-> 1349 return self._build_call_outputs(self._inference_function(*args))
1350 forward_backward = self._select_forward_and_backward_functions(
1351 args,
1352 possible_gradient_type,
1353 executing_eagerly)
1354 forward_function, args_with_tangents = forward_backward.forward()

```

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\polymorphic_function\atomic_function.py:196, in AtomicFunction.__call__(self, *args)

```

194 with record.stop_recording():
195 if self._bound_context.executing_eagerly():
--> 196 outputs = self._bound_context.call_function(
197 self.name,
198 list(args),
199 len(self.function_type.flat_outputs),
200 )
201 else:
202 outputs = make_call_op_in_graph(self, list(args))

```

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\context.py:1457, in Context.call_function(self, name, tensor_inputs, num_outputs)

```

1455 cancellation_context = cancellation.context()
1456 if cancellation_context is None:
-> 1457 outputs = execute.execute(
1458 name.decode("utf-8"),
1459 num_outputs=num_outputs,
1460 inputs=tensor_inputs,
1461 attrs=attrs,
1462 ctx=self,
1463 )
1464 else:
1465 outputs = execute.execute_with_cancellation(

```

```
1466     name.decode("utf-8"),
1467     num_outputs=num_outputs,
1468     (...)
1471     cancellation_manager=cancellation_context,
1472 )
```

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\execute.py:53, in quick_execute(op_name, num_outputs, inputs, attrs, ctx, name)

```
51 try:
52     ctx.ensure_initialized()
--> 53     tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle, device_name, op_name,
54                                           inputs, attrs, num_outputs)
55 except core._NotOkStatusException as e:
56     if name is not None:
```

KeyboardInterrupt:

In []: